

บทที่ 8

ไดเร็กออডিโอ (DirectX Audio)

8.1 ไดเร็กเซาว์ (DirectSound)

ไดเร็กเซาว์ออปเจ็กต์เป็นตัวแทนของอุปกรณ์และถูกใช้ในการจัดการอุปกรณ์และการสร้างเซาว์บัฟเฟอร์ (Sound Buffer)

ไดเร็กเซาว์บัฟเฟอร์ออปเจ็กต์เป็นตัวแทนของบัฟเฟอร์ที่เก็บข้อมูลเสียงเอาไว้ บัฟเฟอร์สามารถที่จะใช้ในการเริ่มต้น, หยุด หรือรักษาดำแหน่ง (pause) ได้ นอกจากนั้นยังกำหนดค่าความถี่หรือรูปแบบได้

เซาว์บัฟเฟอร์ปฐมภูมิ (primary sound buffer) ทำหน้าที่เก็บข้อมูลเสียงที่ผู้ฟังจะได้ยิน ส่วนเซาว์บัฟเฟอร์ทุติยภูมิ (secondary sound buffer) ซึ่งบัฟเฟอร์ชนิดนี้สามารถมีได้หลายตัวโดยแต่ละตัวจะเก็บข้อมูลเสียงหรือสตรีมของเสียง 1 ชุดเอาไว้ ไดเร็กเซาว์จะสร้างบัฟเฟอร์ปฐมภูมิโดยอัตโนมัติ ส่วนบัฟเฟอร์ทุติยภูมิจะสร้างขึ้นโดยแอปพลิเคชัน เมื่อมีการเล่นข้อมูลเสียงในบัฟเฟอร์ทุติยภูมิไดเร็กเซาว์จะผสมข้อมูลเสียงของแต่ละบัฟเฟอร์ทุติยภูมิเข้าด้วยกันแล้วเก็บลงในบัฟเฟอร์ปฐมภูมิแล้วส่งไปยังอุปกรณ์เอาต์พุต เงื่อนไขที่เป็นตัวกำหนดจำนวนของบัฟเฟอร์ทุติยภูมิขึ้นอยู่กับเวลาในการประมวลผลข้อมูลเท่านั้นเนื่องจากไดเร็กเซาว์ไม่มีข้อกำหนดของจำนวนบัฟเฟอร์

เสียงที่มีความยาวไม่มากสามารถที่จะไหลเข้าสู่บัฟเฟอร์ได้ทั้งหมดและสามารถที่จะเล่นเสียงนั้นได้ด้วยการเรียกฟังก์ชันเพียงฟังก์ชันเดียว แต่ถ้าข้อมูลมีความยาวมากเกินไปจะต้องบัฟเฟอร์จะต้องรับข้อมูลในรูปแบบของสตรีม โดยแอปพลิเคชันสามารถรู้ได้ว่าจะต้องส่งข้อมูลเป็นสตรีมข้อมูลเมื่อใดโดยการสำรวจตำแหน่งของตัวชี้ที่แสดงตำแหน่งการเล่นหรือรับการแจ้งเมื่อตัวชี้ตำแหน่งมาถึงตำแหน่งที่กำหนด

รูปแบบของข้อมูลในเซาว์บัฟเฟอร์ทุติยภูมิจะต้องกำหนดโดยแอปพลิเคชันเนื่องจากไดเร็กเซาว์ไม่มีเมทอดที่ช่วยในการแปลงข้อมูลจากไฟล์ wave

8.2 อุปกรณ์ในไดเร็กเซาว์ (DirectSound Devices)

ในการใช้งานไดเร็กเซาว์ขั้นตอนแรกจะต้องสร้างไดเร็กเซาว์ออปเจ็กต์ซึ่งในเป็นตัวแทนของอุปกรณ์เสียง เช่น การ์ดเสียง

8.3 การระบุอุปกรณ์ในไดเร็กเซาว์ (Enumeration of Sound Devices)

อันที่จริงแล้วแอปพลิเคชันไม่จำเป็นต้องระบุอุปกรณ์ก็ได้ถ้าต้องการใช้อุปกรณ์ที่ติดตั้งไว้โดยปริยาย (default device) ด้วยการเรียกฟังก์ชัน DirectSoundCreate8

แต่ถ้าต้องการที่จะสร้างอุปกรณ์อย่างเฉพาะเจาะจงหรือต้องการที่จะให้ผู้ใช้งานสามารถเลือกอุปกรณ์ที่ต้องการจะใช้งานได้ หรือในกรณีที่จะต้องใช้งานอุปกรณ์มากกว่า 1 ตัว ก็มีความจำเป็นที่จะต้องทำการระบุอุปกรณ์ที่มีอยู่ในเครื่องคอมพิวเตอร์ก่อน

การระบุอุปกรณ์ในไดเร็กเซา์มีจุดประสงค์ ดังนี้

1. เพื่อตรวจสอบว่าฮาร์ดแวร์สามารถใช้งานได้หรือไม่
2. เพื่อที่จะกำหนดค่าเฉพาะเจาะจง (Globally unique identifier GUID) ให้กับแต่ละอุปกรณ์
3. สามารถสร้างไดเร็กเซา์ออปเจ็กต์สำหรับแต่ละอุปกรณ์ที่ทำการระบุอยู่แบบชั่วคราวเพื่อใช้ในการตรวจสอบความสามารถของอุปกรณ์ได้

ในการระบุอุปกรณ์จะต้องกำหนดฟังก์ชันเรียกกลับ (callback function) ที่จะถูกเรียกทุกๆ ครั้งที่ไดเร็กเซา์ทำการระบุอุปกรณ์ ซึ่งในฟังก์ชันนี้สามารถที่จะทำการประมวลผลใดๆ ก็ได้ และสามารถกำหนดชื่อฟังก์ชันอย่างไรก็ได้แต่จะต้องอยู่ในรูปแบบที่กำหนดไว้ใน DSEnumCallback โดยฟังก์ชันเรียกกลับนี้จะต้องส่งค่า TRUE ถ้าต้องการให้มีการระบุอุปกรณ์ต่อไป และส่งค่า FALSE ถ้าต้องการให้จบขั้นตอนการระบุอุปกรณ์

ตัวอย่าง ฟังก์ชันเรียกกลับต่อไปนี้ทำการเพิ่มชื่อของอุปกรณ์ในบัญชีของคอมโบบ็อก

```

BOOL CALLBACK DSEnumProc(LPGUID lpGUID, LPCTSTR lpszDesc,
                          LPCTSTR lpszDrvName, LPVOID lpContext )
{
    HWND hCombo = *(HWND *)lpContext;
    LPGUID lpTemp = NULL;
    if ( lpGUID != NULL )
    {
        if (( lpTemp = malloc( sizeof(GUID))) == NULL )
            return( TRUE );

        memcpy( lpTemp, lpGUID, sizeof(GUID));

        ComboBox_AddString( hCombo, lpszDesc );
        ComboBox_SetItemData( hCombo, ComboBox_FindString( hCombo, 0, lpszDesc ), lpTemp );
        return( TRUE );
    }
}

```

8.4 การสร้างไดเร็กเซา์ออปเจ็กต์

วิธีการสร้างไดเร็กเซา์ออปเจ็กต์ที่ง่ายที่สุดคือการเรียกฟังก์ชัน DirectSoundCreate8 ถ้าพารามิเตอร์ตัวแรกคือ ค่า GUID ของอุปกรณ์ที่จะติดต่อกับออปเจ็กต์ ซึ่งค่านี้สามารถหาได้จากขั้นตอนการระบุอุปกรณ์ หรือจะส่งค่าที่มีการกำหนดเอาไว้แล้วดังต่อไปนี้

DSDEVID_DefaultPlayback

ใช้อุปกรณ์ที่กำหนดไว้โดยปริยายโดยระบบ ถ้าต้องการระบุค่าพารามิเตอร์นี้อาจจะส่งค่า NULL เข้าไปแทน

DSDEVID_DefaultVoicePlayback

ค่านี้คือการกำหนดให้ใช้อุปกรณ์ติดต่อเสียง (voice communication device) โดยปริยาย ซึ่งโดยปกติแล้วอุปกรณ์นี้จะเป็นหูฟังแบบ USB ที่มีไมโครโฟน

ถ้าไม่มีอุปกรณ์ที่กำหนด การเรียกฟังก์ชัน DirectSoundCreate8 จะไม่สำเร็จ
ตัวอย่างการสร้างไคเร็กเซว

LPDIRECTSOUND8 lpds;

HRESULT hr = DirectSoundCreate8(NULL, &lpds, NULL);

8.5 ระดับการทำงานร่วมกัน (Cooperative Levels)

เนื่องจากในเวลาขณะหนึ่งอาจจะมีแอปพลิเคชันทำงานอยู่มากกว่า 1 แอปพลิเคชัน การกำหนดระดับการทำงานร่วมกันเป็นการกำหนดให้ไคเร็กเซวควบคุมไม่ให้มีแอปพลิเคชันเข้าใช้อุปกรณ์ในทางที่ผิดหรือผิดเวลา แอปพลิเคชันแต่ละตัวจะมีระดับการทำงานร่วมกันของตัวเองที่จะเป็นตัวกำหนดจำนวนของแอปพลิเคชันอื่นๆ ที่สามารถเข้าใช้งานอุปกรณ์ได้

หลังจากการสร้างออปเจกต์ไคเร็กเซวแล้วจะต้องกำหนดค่าระดับการทำงานร่วมกันโดยเรียกเมทอด IDirectSound8::SetCooperativeLevel ก่อนที่จะเล่นเสียง

HRESULT hr = lpDirectSound->SetCooperativeLevel(hwnd, DSSCL_PRIORITY);

ระดับการทำงานร่วมกันแบบปกติ (Normal Cooperative Level)

ในแบบนี้แอปพลิเคชันไม่สามารถกำหนดรูปแบบของบัพเฟอร์ปฐมภูมิได้ นั่นคือ จะต้องใช้บัพเฟอร์ปฐมภูมิที่มีความถี่ 22 kHz แบบสเตอริโอ และ 8-bit แชมเพิล

ระดับการทำงานร่วมกันแบบไพรอริตี (Priority Cooperative Level)

เมื่อใช้อุปกรณ์ไคเร็กเซวในระดับไพรอริตี แล้วแอปพลิเคชันมีสิทธิที่จะเข้าถึงรีซอร์สบนฮาร์ดแวร์ เช่น ฮาร์ดแวร์มิกซิง (Hardware Mixing), สามารถกำหนดรูปแบบของบัพเฟอร์ปฐมภูมิ

แอปพลิเคชันจำพวกเกมสัควร์ใช้ระดับการทำงานร่วมกันในระดับนี้เนื่องจากสามารถกำหนดการแชนเพิลและความถี่ได้

ระดับการทำงานร่วมกันแบบเขียนบนปฐมภูมิ (Write-primary Cooperative Level)

ระดับนี้เป็นระดับที่แอปพลิเคชันสามารถเข้าถึงบัฟเฟอร์ปฐมภูมิได้โดยตรง นั่นคือ แอปพลิเคชันจะเขียนข้อมูลโดยตรงลงบนบัฟเฟอร์ปฐมภูมิและจะไม่สามารถเล่นเสียงจากบัฟเฟอร์ทุติยภูมิได้

8.6 ความสามารถของอุปกรณ์ (Device Capabilities)

ไคเร็กเซวาร์ับข้อมูลความสามารถของอุปกรณ์เพื่อที่ใช้ข้อมูลนั้นในการปรับความคุณสมบัติต่างๆ ให้ใช้ประโยชน์จากอุปกรณ์ที่มีความสามารถในการเร่งความเร็วได้มากที่สุด หลังจากที่สร้างออปเจกต์ไคเร็กเซวาร์แล้วสามารถรับค่าความสามารถของอุปกรณ์เสียงได้ด้วยการเรียกเมทอด `IdirectSound8::`

`GetCaps`

ตัวอย่าง

`DSCAPS dscaps;`

`dscaps.dwSize = sizeof(DSCAPS);`

`HRESULT hr = lpDirectSound->GetCaps(&dscaps);`

โครงสร้างข้อมูล DSCAPS รับข้อมูลเกี่ยวกับประสิทธิภาพและรีซอร์สของอุปกรณ์ รวมทั้งขนาดของรีซอร์สทั้งหมดและขนาดของรีซอร์สที่สามารถใช้งานได้ในขณะนั้น ถ้าแอปพลิเคชันต้องการจะเพิ่มส่วนการใช้งานฮาร์ดแวร์ควรจะเรียกเมทอดนี้ทุกครั้งก่อนที่จะทำ

8.7 ไคเร็กเซวาร์บัฟเฟอร์

ไคเร็กเซวาร์บัฟเฟอร์ออปเจกต์ควบคุมการส่งข้อมูลเวฟจากต้นทางไปสู่ปลายทาง ต้นทางอาจจะเป็นซินทีไซเซอร์ (synthesizer), บัฟเฟอร์ตัวอื่น, ไฟล์ หรือรีซอร์ส ปลายทางโดยปกติแล้วจะเป็นตัวผสมเสียงในบัฟเฟอร์ปฐมภูมิ และจากบัฟเฟอร์ปฐมภูมิข้อมูลจะถูกส่งไปยังฮาร์ดแวร์ซึ่งจะแปลงค่า PCM แซมเพิลไปเป็นคลื่นเสียง

บัฟเฟอร์

เมื่อไคเร็กเซวาร์เริ่มต้นการทำงานจะสร้างบัฟเฟอร์ปฐมภูมิโดยอัตโนมัติและจัดการกับบัฟเฟอร์ปฐมภูมิสำหรับการผสมเสียงและส่งข้อมูลไปยังอุปกรณ์เอาต์พุต โดยทั่วไปแล้ว บัฟเฟอร์แบบปฐมภูมิจะได้รับการควบคุมจากไคเร็กเซวาร์อยู่เบื้องหลัง ดังนั้น แอปพลิเคชันจึงไม่จำเป็นต้องสนใจความเป็นไปของมัน

แอปพลิเคชันจะต้องสร้างบัฟเฟอร์ทุติยภูมิอย่างน้อย 1 บัฟเฟอร์เพื่อเก็บข้อมูลเสียงที่ต้องการจะเล่น บัฟเฟอร์แบบทุติยภูมินี้สามารถที่จะสร้างขึ้นและให้คงอยู่ตลอดไปหรือทำลายทิ้งเมื่อไม่ต้องการก็ได้ และสามารถเก็บข้อมูลที่ใช้เล่นซ้ำกี่ครั้งก็ได้ หรือจะเขียนข้อมูลใหม่ทุกครั้งที่จะเล่น นอกจากนั้นยังสามารถกำหนดให้การเล่นเป็นแบบครั้งเดียวจบหรือเล่นวนไปเรื่อยๆ ได้เช่นกัน และยังสามารถใช้กับสตรีมในกรณีที่มีขนาดของไฟล์ใหญ่เกินไปได้อีกด้วย

ตำแหน่งของบัฟเฟอร์สามารถอยู่ได้ทั้งในฮาร์ดแวร์และซอฟต์แวร์ ในกรณีของฮาร์ดแวร์การผสมเสียงจะเกิดขึ้นโดยตัวประมวลผลบนการ์ดเสียง ส่วนในกรณีของซอฟต์แวร์จะผสมเสียงโดยใช้ CPU นอกจากนั้น ข้อมูลของซอฟต์แวร์จะอยู่ในแรมโมรีของระบบเสมอ ส่วนฮาร์ดแวร์สามารถอยู่ได้ทั้งในแรมโมรีของระบบและแรมโมรีบนการ์ดเสียง

การผสมเสียงสามารถทำได้โดยการสั่งให้มีการเล่นข้อมูลในบัฟเฟอร์หลายๆ คู่กัน ไม่มี การจำกัดจำนวนของบัฟเฟอร์ที่สามารถเล่นได้ในไคร์เรกเซวล์ ดังนั้นจำนวนของบัฟเฟอร์ที่สามารถเล่นได้ จะขึ้นอยู่กับความสามารถในการประมวลผลเท่านั้น

การสร้างบัฟเฟอร์ทุดิยภูมิ

การสร้างบัฟเฟอร์ทุดิยภูมิทำได้โดยการเรียกเมทอด IDirectSound8::CreateSoundBuffer ซึ่งจะส่งค่าพอยเตอร์ที่ชี้ไปยังอินเทอร์เฟซ IDirectSoundBuffer8 ซึ่งใช้ในการควบคุมและการเล่นบัฟเฟอร์

ตัวอย่าง

```

BOOL AppCreateBasicBuffer( LPDIRECTSOUND8 lpDirectSound,
    LPDIRECTSOUNDBUFFER *lplpDsb)
{
    PCMWAVEFORMAT pcmwf;
    DSBUFFERDESC dsbdesc;
    HRESULT hr;

    // Set up wave format structure.
    memset(&pcmwf, 0, sizeof(PCMWAVEFORMAT));
    pcmwf.wf.wFormatTag = WAVE_FORMAT_PCM;
    pcmwf.wf.nChannels = 2;
    pcmwf.wf.nSamplesPerSec = 22050;
    pcmwf.wf.nBlockAlign = 4;
    pcmwf.wf.nAvgBytesPerSec = pcmwf.wf.nSamplesPerSec * pcmwf.wf.nBlockAlign;
    pcmwf.wBitsPerSample = 16;

    // Set up DSBUFFERDESC structure.
    memset(&dsbdesc, 0, sizeof(DSBUFFERDESC));
    dsbdesc.dwSize = sizeof(DSBUFFERDESC);
    dsbdesc.dwFlags=
        DSBCAPS_CTRLPAN | DSBCAPS_CTRLVOLUME | DSBCAPS_CTRLFREQUENCY;
    dsbdesc.dwBufferBytes = 3 * pcmwf.wf.nAvgBytesPerSec;
    dsbdesc.lpwfxFormat = (LPWAVEFORMATEX)&pcmwf;

```

```

// Create buffer.

hr = lpDirectSound->CreateSoundBuffer(&dsbdesc, lpDsb, NULL);
if SUCCEEDED(hr)
{
    // IDirectSoundBuffer interface is in *lpDsb.

    // Use QueryInterface to obtain IDirectSoundBuffer8.

    return TRUE;
}
else
{
    // Failed.

    *lpDsb = NULL;

    return FALSE;
}
}

```

8.8 การใช้งานข้อมูล Wave

ไดเรกต์ซาว์บ์เฟอร์สามารถเล่นได้เฉพาะข้อมูลเสียงในแบบเวฟ (waveform audio data) เท่านั้น ซึ่งข้อมูลจะประกอบด้วยค่าดิจิทัลแซมเพิล (Digital sample) ของเสียงในอัตราแซมเพิลที่คงที่ ซึ่งการแทนสัญญาณแบบบิตสตรีมด้วยตัวเลขที่เรียงเป็นลำดับเรียกว่า pulse code modulation (PCM)

ข้อมูลเวฟโดยปกติจะเก็บอยู่ในรูปของไฟล์หรือรีซอร์สใน Resource Interchange File Format (RIFF) ข้อมูลจะเก็บส่วนที่บรรยายรูปแบบของข้อมูล รวมทั้งพารามิเตอร์ต่างๆ เช่น อัตราแซมเพิลหรือจำนวนของช่องทางเอาต์พุต (output channel) รูปแบบของเสียงบางรูปแบบสามารถใช้โครงสร้างข้อมูล WAVEFORMATEX หรือ WAVEFORMATEXTENSIBLE ในการอธิบายได้

การอ่านข้อมูลเวฟจากไฟล์

เนื่องจากไดเรกต์ซาว์บ์ไม่ได้สร้างเมทอดสำหรับการอ่านไฟล์เอาไว้ให้ แต่ใน Dsutil.cpp ที่มาพร้อมกับ SDK ได้สร้างคลาส CwaveFile ไว้ให้แล้วซึ่งมีฟังก์ชันการทำงานดังนี้

Open เปิดไฟล์สำหรับการอ่านและรับข้อมูลรูปแบบของไฟล์หรือเปิดไฟล์สำหรับการเขียนและเขียนข้อมูลในส่วนของหัวไฟล์

GetSize หาขนาดของข้อมูล สามารถเรียกได้หลังจากการเปิดไฟล์

Read คัดลอกข้อมูลไปยังบัฟเฟอร์และเลื่อนตัวชี้ตำแหน่งการอ่านข้อมูล

Write เขียนข้อมูลจากบัฟเฟอร์และเลื่อนตัวชี้ตำแหน่งของการเขียนข้อมูล

ResetFile เลื่อนตัวชี้ทั้งการอ่านและเขียนข้อมูลกลับมาที่จุดเริ่มต้น

Close ปิดไฟล์